Dataclass

What is a Dataclass?

A **dataclass** is a special kind of class in Python (introduced in **Python 3.7**) that is designed mainly for **storing data**.

It automatically generates common "**boilerplate**" methods for you, such as:

- **__init__()** (constructor)
- **__repr__()** (nice object printing)
- **__eq__()** (equality comparison)

Syntax
`from dataclasses import dataclass`

If your class mostly contains **attributes** (data) and you don't want to manually write repetitive code, use **@dataclass**.

Why Dataclasses are Useful

Without dataclasses:

Python

```python
class Student:
    def __init__ (self, name,
roll):
        self.name = name
        self.roll = roll
    def __repr__ (self):
        return
f"Student(name={self.name},
roll={self.roll})"
```

With dataclasses:

Python

```python
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    roll: int
```

Less code • More readable • Fewer mistakes
Better default printing & comparisons
Standard structure for “data objects”

Default Values

- You can give a default value to a field.

Python

```python
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    roll: int = 1

a = Student("Ravi")
print(a.roll)  # 1
```

Default Values – VERY IMPORTANT

- **Don't use [] or {} as default directly**

Python

```python
from dataclasses import dataclass

@dataclass
class TeamBad:
    members: list = []    # wrong
```

Because this list can become **"shared"** between objects.

Default Values – VERY IMPORTANT

- **Correct Way – use `field(default_factory)`**

Python

```python
from dataclasses import dataclass, field

@dataclass
class TeamGood:
    members: list = field(default_factory=list)
```

Simple meaning:
"Whenever a new object is created, give it a fresh new list."

Default Values – VERY IMPORTANT

- **DEMO**

Python	Python

`from dataclasses import dataclass`
 	
`@dataclass`
`class TeamBad:`
`  members: list = []  # wrong`
 	
`a = TeamBad()`
`b = TeamBad()`
`a.members.append("Aman")`
`print("Bad A:", a.members)  # ['Aman']`
`print("Bad B:", b.members)  # ['Aman']<- shared!`
	`from dataclasses import dataclass, field`
	
	`@dataclass`
	`class TeamGood:`
	`  members: list = field(default_factory=list)`
	
	`x = TeamGood()`
	`y = TeamGood()`
	`x.members.append("Aman")`
	`print("Good X:", x.members)  # ['Aman']`
	`print("Good Y:", y.members)  # []<- separate!`

__post_init__()

- Sometimes after creating the object, you want to run **extra logic** like validation.
- **Dataclass** gives you **__post_init__()** for that.

Python

```python
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int

    def __post_init__(self):
        if self.age < 0:
            raise ValueError("Age cannot be negative")
```

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Packages